Worksheet 5 — Cross-Site Scripting & Client-Side Risks (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 5 Aligned:** OWASP 2025 **A05 Injection** · **CWE-79** (XSS), **CWE-352** (CSRF), **CWE-1004** (cookie without HttpOnly) **Signature game:** ⛳ **XSS Golf** — fire `alert(1)` in the fewest characters possible. Lower payload length = lower score = better. Par for reflected is the `<img>` vector; can you go under par?

> ⚠️ **Ethics note:** Use only the provided `vulnerable_app.py` sandbox and your own Juice Shop container. Stealing real users' cookies or sessions is illegal. All "session theft" steps here target the sandbox cookie `session=abc123` only.

Part 1 — Student Information

Name	Student ID	Date	Group
Athichon kaewla	6631503046	11/9/2569	—

AI disclosure: Used AI (ChatGPT and Claude) to check my answers and explain concepts; all payloads were run and screenshots were taken by me.

Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Distinguish **reflected**, **stored**, and **DOM-based** XSS by *where* the untrusted data is injected and *when* it executes. Which two does our `vulnerable_app.py` implement, and at which routes?

- Reflected XSS involves malicious data from a request being immediately reflected and executed. Stored XSS occurs when a payload is saved on the server and executed later when someone visits the page. DOM-based XSS happens when client-side JavaScript inserts unsafe data into the DOM. Our application (vulnerable_app.py) only covers Reflected and Stored XSS; DOM XSS is merely an optional target.

2. How does **contextual output encoding** (`markupsafe.escape`) stop `<script>` from executing? Why is HTML-context encoding different from JavaScript- or URL-context encoding?

- Context-aware output encoding prevents XSS by converting special characters into safe text—for instance, `markupsafe.escape()` converts `<` into `&lt;`, causing `<script>` to be displayed as text rather than executed. Encoding must be tailored to the specific context because HTML, JavaScript, and URLs have different syntaxes.

3. Explain how a strict **Content-Security-Policy** (`script-src 'self'`) defeats an *injected* inline script even when encoding is missing.

- A strict CSP configured with `script-src 'self'` will block injected inline scripts from executing; the browser will block them even if output encoding fails. However, CSP serves only as an additional layer of defense and is not a substitute for proper encoding.

4. What do the cookie flags **HttpOnly**, **SameSite**, and **Secure** each protect against? Map each to a concrete attack (cookie theft via XSS, CSRF, network sniffing).

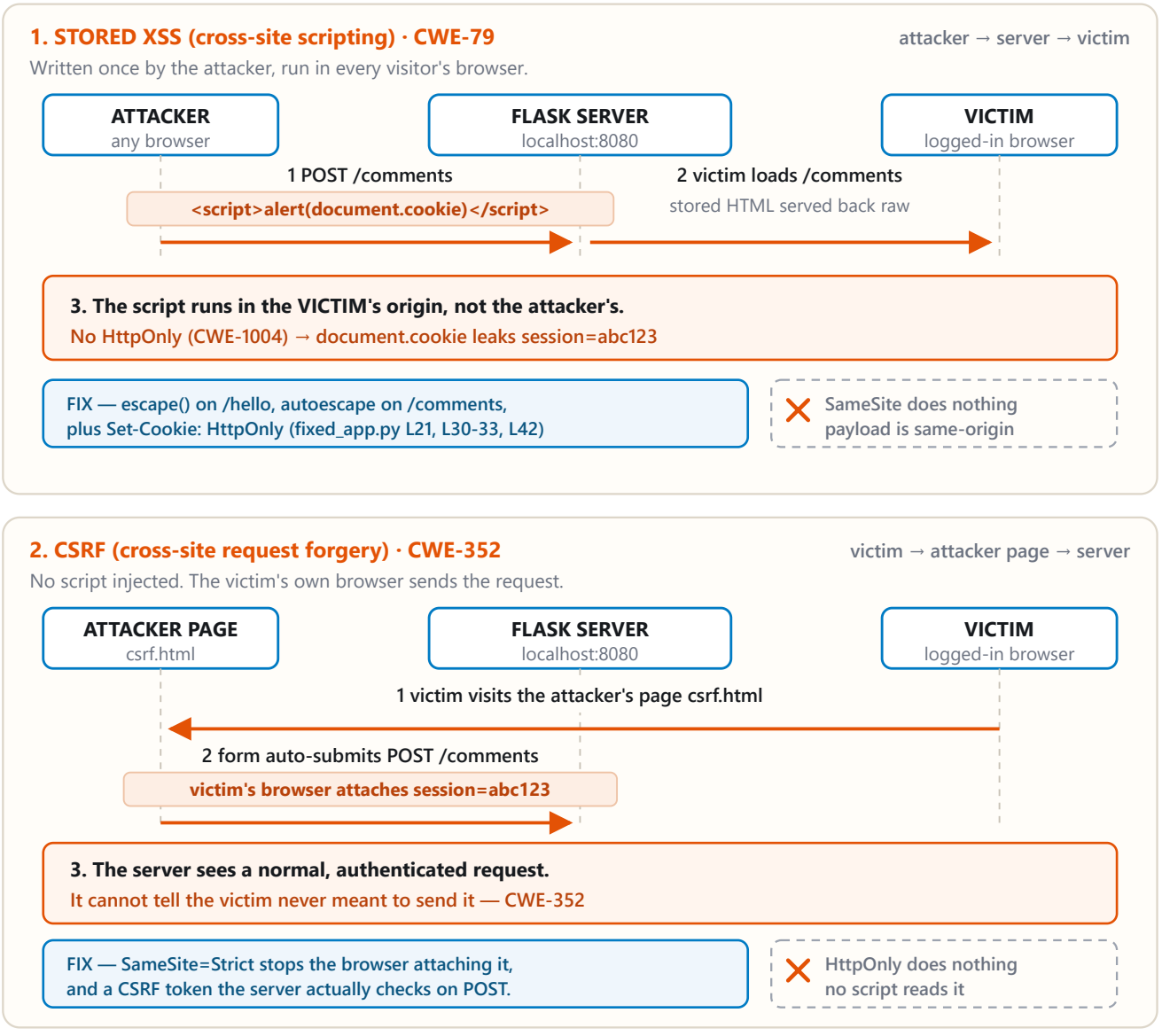
- HttpOnly prevents JavaScript from reading cookies, helping to mitigate cookie theft via XSS. SameSite restricts cookies from being sent with cross-site requests, helping to prevent CSRF. Secure ensures that cookies are sent only over HTTPS, protecting them from network sniffing

5. Why does **CSRF** (CWE-352) work even without any script injection, and how does `SameSite=Strict` plus the same-origin policy blunt it?

- CSRF does not require script injection because the victim's browser automatically attaches the session cookie to a forged request. The Same-Origin Policy prevents an attacker from reading cross-origin responses, but it does not prevent cross-origin requests from being sent. SameSite=Strict helps prevent CSRF by stopping cookies from being sent with cross-site requests

Part 3 — Hands-on Lab (150 min)

Same browser, same cookie, opposite directions



- fixed_app.py adds encoding, CSP and HttpOnly — the XSS dies.
- The CSRF still works: /comments checks no token on POST.

Learning goals: land reflected + stored XSS, abuse a JS-readable cookie, build a CSRF PoC against the comment board, then prove `fixed_app.py` blocks all of it.

Prerequisites: Docker + Docker Compose, a browser with DevTools, a text editor. Working dir: `labs/week05-xss-client-side/`.

Environment setup

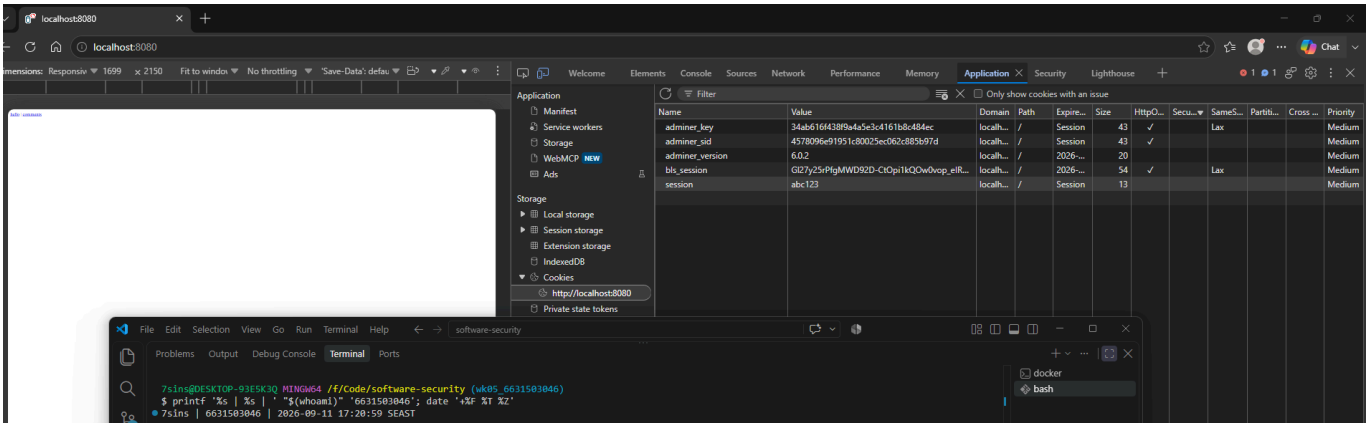
```
cd labs/week05-xss-client-side
docker compose up            # python:3.12-slim + flask, runs vulnerable_app.py
# vulnerable app -> http://localhost:8080   (service name: xss-lab, port 8080)
```

Optional secondary target (for DOM XSS, which our app does not expose):

```
docker run --rm -p 3000:3000 bkimminich/juice-shop       # ->
http://localhost:3000
```

What to submit per task: the exact **payload**, a **screenshot** of the alert/effect, and a **2–3 sentence mitigation**.

Task 0 — Onboarding (5 min). Browse `http://localhost:8080/`. Open DevTools → Application → Cookies and confirm `session=abc123` is set with **no HttpOnly / SameSite**. Screenshot it. *Deliverable: screenshot.*

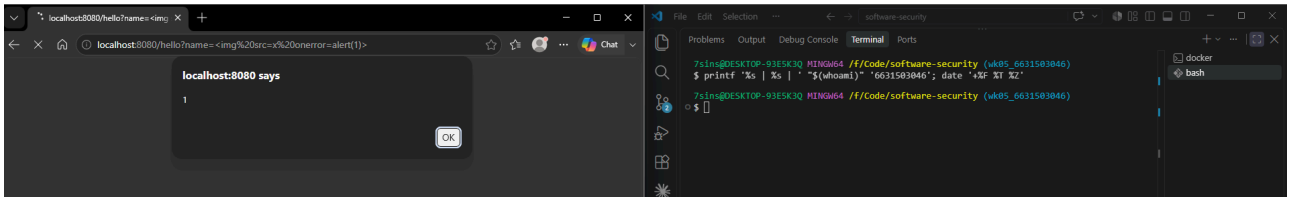


Task 1 — Reflected XSS + XSS Golf (30 min) ⛳.

- *Goal:* execute JS via `/hello`, then minimize the payload.
- *Steps:* visit `/hello?name=<script>alert(1)</script>`, then the alternate `/hello?name=<img src=x onerror=alert(1)>` (useful when `<script>` tags specifically are filtered — note it's actually 3 characters longer, not shorter). Record each payload's character count for your golf score.

- *Deliverable:* both payloads + char counts + screenshot of `alert(1)` + your lowest score. "

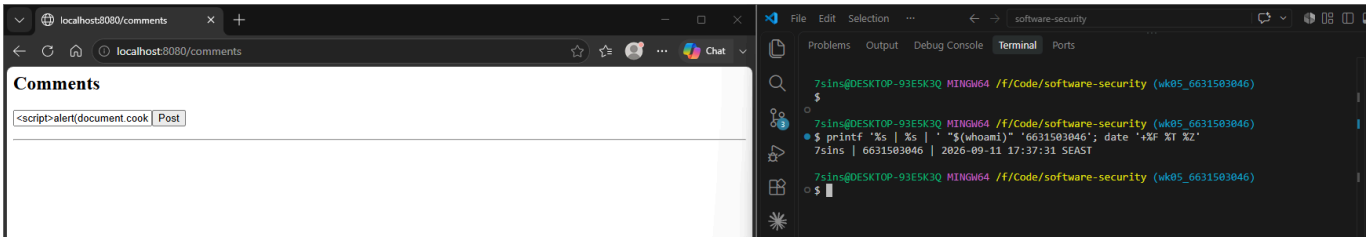
<script>alert(1)</script>" 25 Length "" 28 Length "

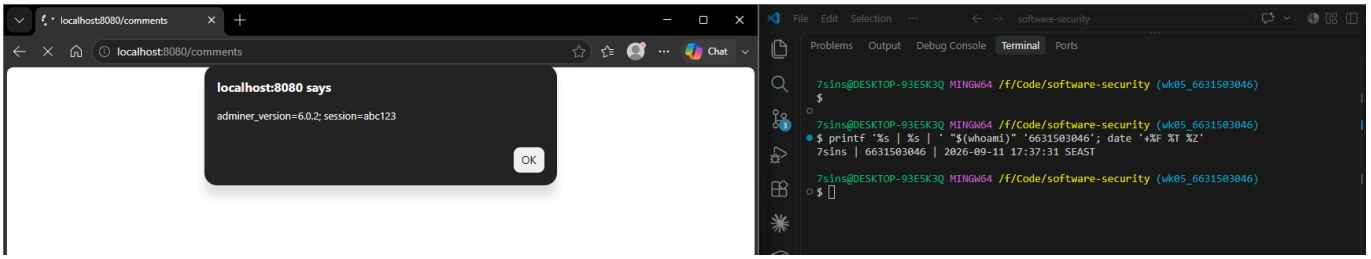


Lowest score: 21

Task 2 — Stored XSS (30 min) ⛳.

- *Goal:* persist a script that runs for every visitor of `/comments`.
- *Steps:* POST a comment with body `<script>alert(document.cookie)</script>` (use the form or `curl -d 'body=...'`). Reload `/comments` and watch the cookie pop.
- *Deliverable:* payload + screenshot of the alert showing `session=abc123` + why stored XSS is more dangerous than reflected.



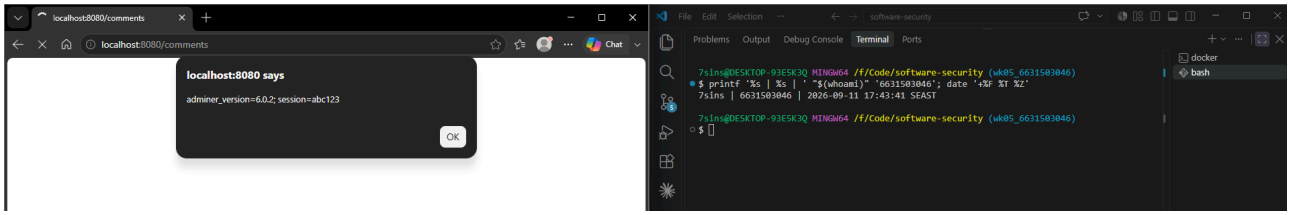


Stored XSS is generally more dangerous than reflected XSS because the malicious script is stored on the server and can execute automatically for multiple users who access the affected page

Task 3 — Cookie theft via XSS (25 min).

- *Goal:* show the cookie is readable by injected JS because **HttpOnly is missing** (CWE-1004).
- *Steps:* store `<script>new Image().src='http://localhost:8080/hello?name='+document.cookie</script>` (a beacon), or simply `<img src=x onerror=alert(document.cookie)>`. Observe the cookie value being exfiltrated/displayed.

- *Deliverable:* payload + screenshot + 2–3 sentences on how HttpOnly would have stopped this.



  Without HttpOnly, injected JavaScript can read the session cookie through document.cookie and steal it. With HttpOnly enabled, JavaScript cannot access the cookie, preventing cookie theft. However, HttpOnly does not prevent XSS itself, so output encoding is still required

Task 4 — CSRF PoC (30 min).

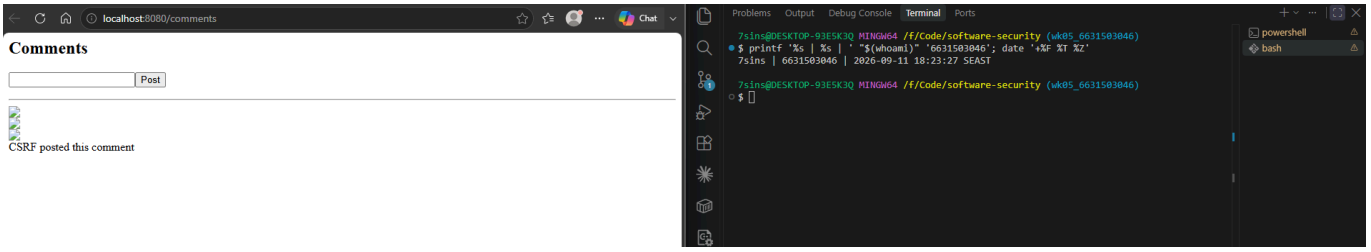
- *Goal:* make a third-party page force a state-changing POST to `/comments`.
- *Steps:* create a local `csrf.html` with an auto-submitting form targeting the board (no token exists, cookie has no SameSite, so the browser attaches `session` cross-site):

```
<body onload="document.forms[0].submit()">
  <form action="http://localhost:8080/comments" method="POST">
    <input name="body" value="CSRF posted this comment">
  </form>
</body>
```

  Open the file and confirm the comment appears on `/comments`.
- *Deliverable:* the HTML + screenshot of the forged comment + why `SameSite=Strict` blocks it.

```
xss-context
```

CSRF posted this comment



- CSRF works because the browser automatically attaches the session cookie to a cross-site POST request. With SameSite=Strict, the browser does not send the cookie when the request comes from another site. As a result, the forged POST reaches the server without the victim's session, so the server cannot treat the request as coming from the logged-in victim

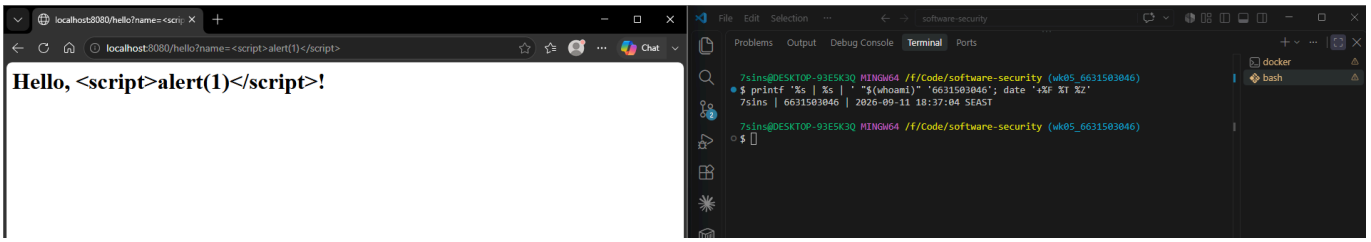
Task 5 — Defend / fix it (30 min) 🛡️.

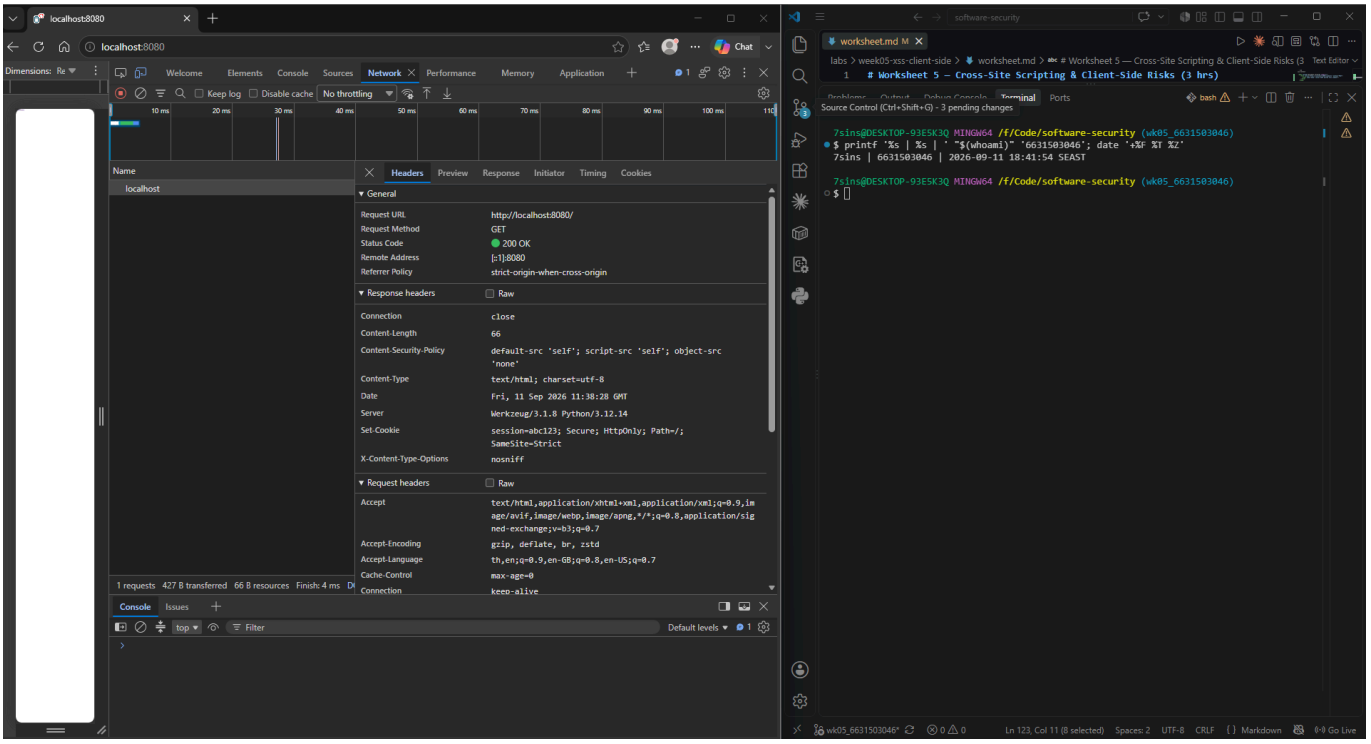
- *Goal:* prove `fixed_app.py` blocks Tasks 1–3, then show that Task 4's CSRF PoC still gets through and explain why.
- *Steps:* stop the vulnerable container (`Ctrl-C`), then:

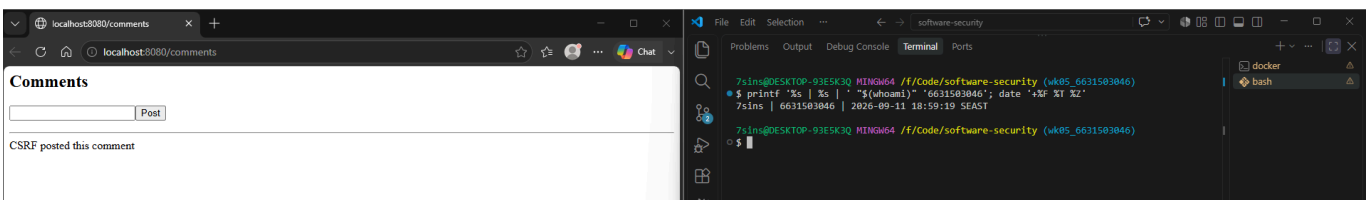
```
docker compose run --rm --service-ports xss-lab bash -c "pip install --no-cache-dir flask && python fixed_app.py"
```

Re-fire each payload. Expected: `/hello` renders the script **as text** (escape, L21), stored comments render literally (Jinja autoescape, L30–33), a strict CSP header is now present as defense-in-depth (`Content-Security-Policy: script-src 'self'`, L12 — check DevTools → Network → Response Headers; escaping already neutralizes these payloads, so no CSP *violation* fires in the console), and the cookie now has `HttpOnly; SameSite=Strict; Secure` (L42). Then re-run Task 4's `csrf.html` PoC against `fixed_app.py`: it **still posts the forged comment** — `/comments` (L25–28) never checks the `session` cookie or a CSRF token before accepting a POST, so hardening the cookie only stops the browser from *attaching* it cross-site; it doesn't stop the request itself from being processed.

- *Deliverable:* screenshots of escaped output + the CSP response header + the hardened cookie flags + the still-successful Task 4 forgery against `fixed_app.py`, with 2–3 sentences on why cookie hardening alone doesn't close CSRF here (no server-side check tied to the cookie, and no CSRF token).







Even though fixed_app.py uses SameSite=Strict, cookie hardening alone is not enough because /comments processes the POST request without checking the session or a CSRF token. Therefore, a forged request can still

be processed and post a comment. To fully prevent CSRF, the server must validate a CSRF token tied to the user's session before making the state change

Part 4 — Reflection

1. **CWE/OWASP mapping:** map your reflected/stored XSS to **CWE-79** and your CSRF PoC to **CWE-352**, both under OWASP 2025 **A05 Injection** (CSRF historically A01/A05).

- Reflected and stored XSS map to CWE-79, while the CSRF PoC maps to CWE-352. Both are mapped to OWASP 2025 A05 Injection in this task

2. **Real breach:** the **2018 British Airways breach** (~380k payment records) used malicious JavaScript (Magecart) injected into the site to skim card data — a client-side script-injection failure. In 3–4 sentences relate it to this lab's XSS and CSP lessons.

- The British Airways breach involved malicious JavaScript running on the website and stealing payment data. This relates to our XSS lab because untrusted script executes in the user's browser. Output encoding helps prevent injected data from becoming executable code, while a strict CSP provides an additional layer by restricting which scripts the browser can execute

3. **Best mitigation:** between output encoding, a strict CSP, and HttpOnly+SameSite cookies, which gives the broadest defense-in-depth, and why is "encoding alone" still risky?

- Contextual output encoding is the primary root-cause fix, but a strict CSP gives the broadest defense-in-depth because it is a single global policy that blocks injected scripts even when encoding is missed. Encoding alone is risky because it must be applied correctly at every output point — a single missed spot means XSS with no backstop — whereas CSP catches what encoding misses

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.

- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide:* `SUBMISSION.md` *in the repo root).*

- **Explain in your own words** *(graded on your reasoning, not copied text):*

    1. What did you do, and **why did the vulnerability work**?
    - I tested reflected XSS at /hello and stored XSS at /comments, read the session cookie using document.cookie, and used csrf.html to send a forged POST request. The XSS worked because user input was inserted directly into HTML without escaping, allowing the browser to interpret it as executable code (CWE-79). The cookie could be read because HttpOnly was not enabled, while the CSRF attack worked because there was no SameSite protection or CSRF token, allowing the forged request to be accepted (CWE-352).
    2. **Why does your fix actually stop it** — and what could still break it?
    - The fix uses escape() or Jinja autoescaping to make the payload render as text instead of executable code. CSP with script-src 'self' provides an additional layer of protection, while HttpOnly and SameSite=Strict harden the session cookie. However, CSRF can still work because /comments does not validate a CSRF token on the server side, and XSS can return if output encoding is missed at any output point

---

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

1. Prompt asked to the AI (Claude): How should I fix reflected and stored XSS in a Flask application where user input is inserted directly into HTML?

AI's full answer (Claude): The root cause is that you're concatenating untrusted input directly into HTML, so the browser parses it as markup. The fix is contextual output encoding: stop building HTML by hand and let the templating layer escape for you — use Jinja2 with autoescaping (`render_template_string` with `{{ value }}` placeholders) or wrap values in `markupsafe.escape()`, so `<`, `>`, `&` become entities and render as text. This closes both reflected (`/hello`) and stored (`/comments`) XSS. Never pass untrusted data through `|safe` or `Markup()`. As defense-in-depth, add a strict `Content-Security-Policy` (`script-src 'self'`) so an injected inline script won't run even if an escape is missed, and set the session cookie `HttpOnly` so JavaScript can't read it if an XSS slips through.

2. What is wrong / incomplete (quoted line): "set the session cookie `HttpOnly` so JavaScript can't read it if an XSS slips through"

The XSS advice itself is correct, but the answer stops at XSS and cookie flags and **never mentions CSRF**. `/comments` still appends any POST without checking a CSRF token, so the forged cross-site request from Task

4 still succeeds even after all of these fixes — exactly what I proved against `fixed_app.py` in Task 5. Cookie flags reduce cookie theft and cross-site attachment, but they do not stop a tokenless forged POST from being processed.

3. Correct / verified version + why the AI was insufficient:

Keep the output encoding (Jinja autoescaping / `markupsafe.escape`) and the CSP from the AI answer — those correctly fix the XSS. In addition, add a **server-side CSRF defense**: a CSRF token (e.g. Flask-WTF `CSRFProtect`, or a per-session random token in a hidden form field) that `/comments` validates before appending to `COMMENTS`, rejecting missing/invalid tokens with HTTP 400.

The AI's answer fixes the XSS correctly with output encoding but is incomplete because it never adds a CSRF token. As I showed in Task 5, hardening the cookie does not stop `/comments` from accepting a forged POST, so the CSRF hole stays open. A complete fix must validate a server-side CSRF token tied to the session before the state change.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

- The vulnerable app takes whatever the user types — the name in `/hello` or a comment in `/comments` — and pastes it straight into the page's HTML without turning special characters like `<` and `>` into plain text. Because of that, the browser reads the input as real HTML tags and runs any script inside them instead of showing it as text. That is why a payload like `<script>alert(1)</script>`, or an `<img>` with an `onerror` handler, executes in the victim's browser.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Final prompt: Rewrite this Flask app so that reflected XSS at `/hello` and stored XSS at `/comments` are fixed with contextual output encoding (Jinja autoescaping or `markupsafe.escape()`), add a `Content-Security-Policy` header of `script-src 'self'`, and set the session cookie with `HttpOnly`, `SameSite=Strict`, and `Secure`. Give the complete file.

Verified result: I verified this with `fixed_app.py`, which applies exactly these changes. Re-firing the Task 1–3 payloads: `/hello?name=<script>alert(1)</script>` now renders as literal text with no alert, and a stored `<script>` comment displays as text instead of running (see Task 5 screenshots). The exploit now **fails**, so the fix works. Note: CSRF still succeeds, because this prompt only targeted XSS — closing that would need a server-side CSRF token.